AWS Load Balancer — Complete Notes

📌 1. What is a Load Balancer?

Definition

A Load Balancer is a networking service that distributes incoming traffic across multiple targets (EC2 instances, containers, IP addresses, Lambda functions) in one or more Availability Zones to ensure:

- High Availability
- Fault Tolerance
- Scalability
- Reliability

🍽️ Real-World Analogy

Imagine a restaurant with multiple billing counters.

A manager stands at the entrance and sends customers to the counter with the **shortest queue**.

That manager is exactly like a Load Balancer

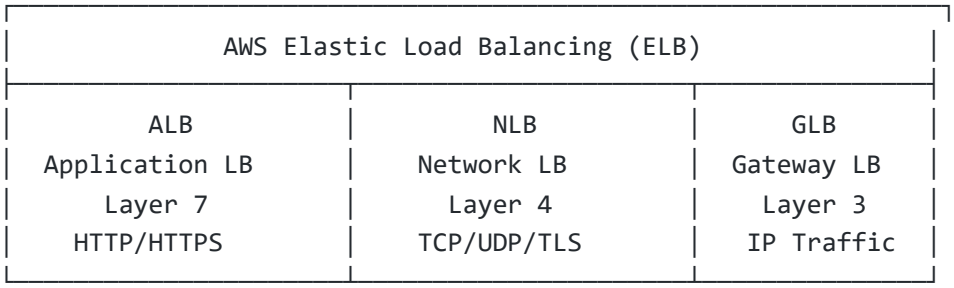
Result:

- No single counter gets overloaded
- Customers are served faster
- If one counter fails, others continue working

✅ Key Benefits of Load Balancers

1. High Availability: Distributes traffic across multiple Availability Zones
2. Fault Tolerance: Stops sending traffic to unhealthy servers
3. Scalability: Handles increasing traffic automatically
4. Security: Backend servers are hidden behind the LB
5. Health Checks: Continuously monitors target health
6. SSL/TLS Termination: Offloads HTTPS encryption/decryption
7. Better Performance: Prevents server overload

🌐 `AWS Elastic Load Balancing (ELB) Family`



> **Note:** Classic Load Balancer (CLB) is legacy/deprecated. AWS recommends ALB or NLB for new projects.

📌 2. Application Load Balancer (ALB) — Layer 7

Definition

An **Application Load Balancer (ALB)** operates at **OSI Layer 7 (Application Layer)**.

It makes routing decisions based on the **content of HTTP/HTTPS requests** such as:

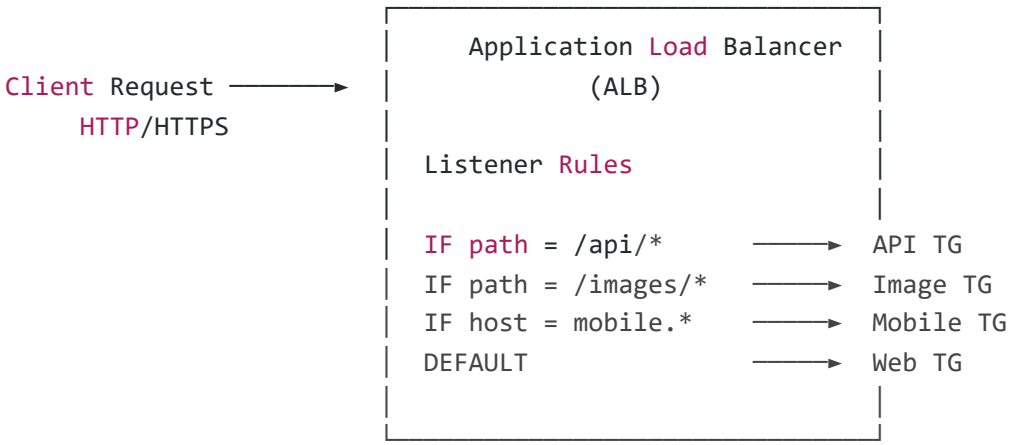
- URL Path
- Hostname
- HTTP Headers
- Query Strings
- HTTP Methods

🔹 ALB Key Characteristics

Feature	Description
Protocol Support	HTTP, HTTPS, gRPC, WebSocket
OSI Layer	Layer 7 (Application Layer)
Routing Type	Content-Based Routing
Targets Supported	EC2, ECS, Lambda, IP addresses
SSL Termination	Supported
Sticky Sessions	Supported

Feature	Description
Health Checks	HTTP/HTTPS path-based
Cross-Zone LB	Enabled by default (free)

🏗️ ALB Architecture



🔧 ALB Core Components

Component	Description
Listener	Checks incoming requests on a specific port
Listener Rule	Defines how traffic should be routed
Target Group	Group of backend servers/targets
Health Check	Monitors target health status

📌 ALB Listener Rules — One-Line Definitions

Rule Type	One-Line Definition
Path-Based Routing	Routes traffic based on URL path
Host-Based Routing	Routes traffic based on domain/hostname

Rule Type	One-Line Definition
Header-Based Routing	Routes traffic based on HTTP headers
Query String Routing	Routes traffic using query parameters
HTTP Method Routing	Routes based on GET, POST, PUT, DELETE methods
Source IP Routing	Routes traffic based on client IP range

🔹 ALB Routing Examples

1. Path-Based Routing

```
example.com/api/*       ⟶ API Servers
example.com/images/*    ⟶ Image Servers
example.com/admin/*     ⟶ Admin Servers
```

2. Host-Based Routing

```
api.example.com         ⟶ API Target Group
www.example.com         ⟶ Web Target Group
admin.example.com       ⟶ Admin Target Group
```

3. Header-Based Routing

```
Header: Device=Mobile   ⟶ Mobile Backend
Header: Device=Web      ⟶ Web Backend
```

4. Query String Routing

```
?platform=mobile        ⟶ Mobile Servers
?version=v2             ⟶ Version 2 Backend
```

5. HTTP Method Routing

```
GET Requests        ⟶ Read Servers
POST Requests       ⟶ Write Servers
```

6. Source IP Routing

```
Corporate IP Range  ⟶ Internal Application
Public Users        ⟶ Public Backend
```

📌 3. Internet-Facing ALB vs Internal ALB

🌍 Internet-Facing ALB

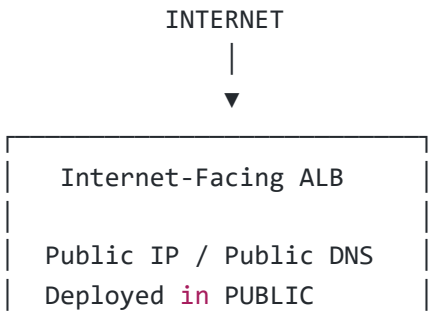
Definition

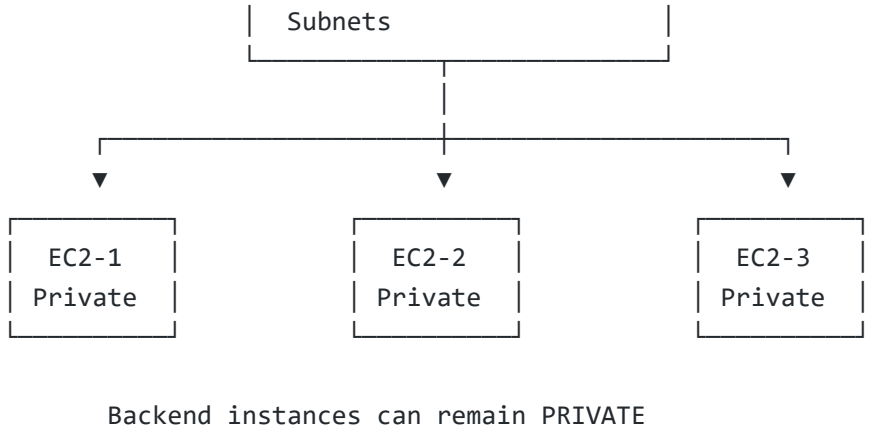
An **Internet-Facing ALB** has:

- Public IP addresses
- Public DNS name
- Internet accessibility

It receives traffic directly from the internet.

🏗️ Architecture





✅ Key Points

Feature	Details
DNS Name	Resolves to Public IPs
ALB Placement	Public Subnets
Targets	Usually Private Subnets
Security Group	Allow 80/443 from 0.0.0.0/0
Accessed By	Public Internet Users
Use Cases	Websites, Public APIs

🔒 Internal ALB

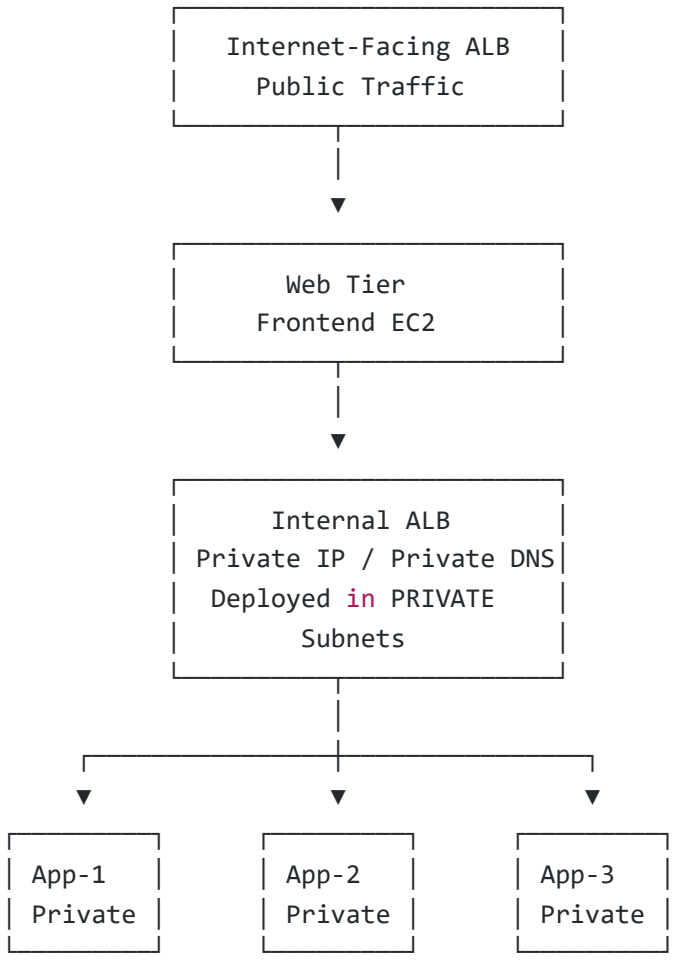
Definition

An **Internal ALB** has:

- Private IP addresses only
- Private DNS name
- No internet accessibility

Used for internal communication inside a VPC.

Architecture



Key Points

Feature	Details
DNS Name	Resolves to Private IPs
Placement	Private Subnets
Access	Only inside VPC
Security Group	Allow VPC CIDR or specific SGs
Use Cases	Microservices, Internal APIs

Internet-Facing ALB vs Internal ALB

Feature	Internet-Facing ALB	Internal ALB
IP Type	Public IP	Private IP
DNS Resolution	Public DNS	Private DNS
Subnets	Public	Private
Accessible From	Internet + VPC	VPC Only
Security Group	0.0.0.0/0	VPC CIDR / Specific SG
Position	Edge Entry Point	Between Application Tiers
Example	www.amazon.com	order-service.internal

📌 4. Network Load Balancer (NLB) — Layer 4

Definition

A **Network Load Balancer (NLB)** operates at **OSI Layer 4 (Transport Layer)**.

It routes traffic based on:

- IP Address
- TCP/UDP Port

It does NOT inspect request content.

Designed for:

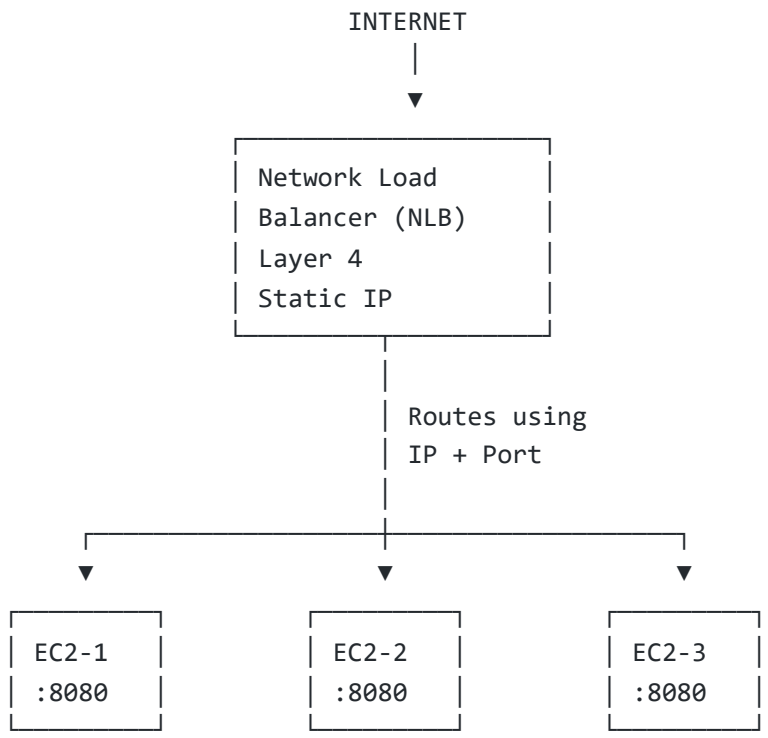
- Ultra High Performance
- Very Low Latency
- Millions of requests per second

🔹 NLB Key Characteristics

Feature	Description
Protocol Support	TCP, UDP, TLS
OSI Layer	Layer 4

Feature	Description
Routing Type	IP + Port Based
Performance	Millions of requests/sec
Latency	Microseconds
Static IP	Supported
Elastic IP	Supported
Preserve Client IP	Yes
SSL Termination	Supported
Health Checks	TCP/HTTP/HTTPS

🏗️ NLB Architecture



📌 Why Static IP in NLB Matters

ALB

```
my-alb-123.elb.amazonaws.com
→ Dynamic IPs
→ IPs can change anytime
→ Use DNS name only
```

NLB

```
52.10.20.30
→ Static IP
→ Never changes
→ Can attach Elastic IP
→ Easy firewall whitelisting
```

📌 5. ALB vs NLB — Detailed Comparison

Feature	ALB	NLB
OSI Layer	Layer 7	Layer 4
Protocols	HTTP, HTTPS, gRPC, WebSocket	TCP, UDP, TLS
Routing	Content-Based	IP + Port
Performance	High	Extreme
Latency	Milliseconds	Microseconds
Static IP	❌ No	✅ Yes
Elastic IP	❌ No	✅ Yes
Preserve Client IP	Via X-Forwarded-For	Native
SSL Termination	Supported	Supported
Sticky Sessions	Cookie-Based	Source IP Based
Lambda Support	✅ Yes	❌ No
ALB as Target	❌	✅
Cross-Zone LB	Enabled by Default	Disabled by Default
Security Groups	Supported	Not Supported

⚠️ Important NLB Security Note

NLB does NOT have Security Groups.

So the backend EC2 instance sees the **real client IP**.

That means:

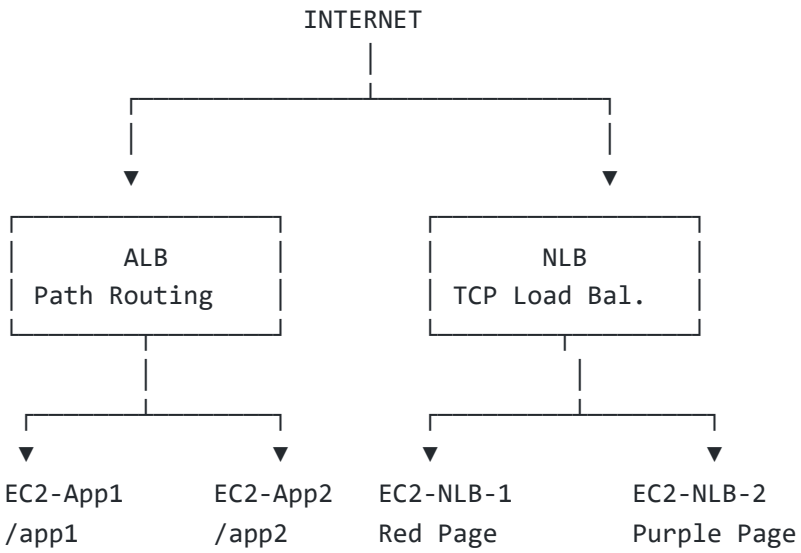
- EC2 Security Group must allow client IP ranges directly
- Traffic does NOT appear from NLB IPs

AWS Load Balancer Practice Tasks (ALB + NLB)

📌 Tasks Overview

In this the following tasks were completed:

🏗️ Architecture Used



📌 Task 1 — Launch EC2-App1 Instance

Objective

Create an EC2 instance that serves the `/app1` application.

Configuration

Setting	Value
Name	EC2-App1
AMI	Ubuntu
Instance Type	t2.micro
VPC	LB-Practice-VPC
Subnet	Public-Subnet-1
Public IP	Enabled
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/app1

cat <<EOF > /var/www/html/app1/index.html
<html>
<body style='background-color:#4CAF50; text-align:center;'>
<h1 style='color:white; margin-top:200px;'>
Welcome to APP 1 🟢
</h1>

<h2 style='color:white;'>
This is EC2 - App1 Server
</h2>

<h3 style='color:white;'>
```

```
Path: /app1
</h3>
</body>
</html>
EOF
```

📌 Task 2 — Launch EC2-App2 Instance

Objective

Create another EC2 instance that serves the `/app2` application.

Configuration

Setting	Value
Name	EC2-App2
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/app2

cat <<EOF > /var/www/html/app2/index.html
<html>
<body style='background-color:#2196F3; text-align:center;'>
<h1 style='color:white; margin-top:200px;'>
```

```
Welcome to APP 2 🔵
</h1>

<h2 style='color:white;'>
This is EC2 - App2 Server
</h2>

<h3 style='color:white;'>
Path: /app2
</h3>
</body>
</html>
EOF
```

📌 Task 3 — Launch EC2-NLB-1 Instance

Objective

Create backend server 1 for Network Load Balancer testing.

Configuration

Setting	Value
Name	EC2-NLB-1
AMI	Ubuntu
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2
```

```
cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#FF5722; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
NLB Server 1 🔴
</h1>

<h2 style='color:white;'>
This request was handled by NLB-Server-1
</h2>

</body>
</html>
EOF
```

📌 Task 4 — Launch EC2-NLB-2 Instance

Objective

Create backend server 2 for Network Load Balancer testing.

Configuration

Setting	Value
Name	EC2-NLB-2
AMI	Ubuntu
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data Script

```
#!/bin/bash

apt update -y
apt install apache2 -y
```

```
systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#9C27B0; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
NLB Server 2 🟣
</h1>

<h2 style='color:white;'>
This request was handled by NLB-Server-2
</h2>

</body>
</html>
EOF
```

📌 Task 5 — Create Target Group for App1

Objective

Create a target group for the App1 server.

Configuration

Setting	Value
Target Group Name	TG-App1
Protocol	HTTP
Port	80
Target Type	Instances
Health Check Path	/app1/index.html

Registered Target

```
EC2-App1
```

📌 Task 6 — Create Target Group for App2

Objective

Create a target group for the App2 server.

Configuration

Setting	Value
Target Group Name	TG-App2
Protocol	HTTP
Port	80
Target Type	Instances
Health Check Path	/app2/index.html

Registered Target

```
EC2-App2
```

📌 Task 7 — Create Application Load Balancer (ALB)

Objective

Create an Internet-Facing ALB for path-based routing.

Configuration

Setting	Value
Name	My-ALB
Type	Application Load Balancer
Scheme	Internet-Facing
Protocol	HTTP
Port	80
VPC	LB-Practice-VPC
Subnets	Public-Subnet-1 & Public-Subnet-2
Security Group	LB-SG

Default Action

```
Forward traffic to TG-App1
```

📌 Task 8 — Configure Path-Based Routing

Objective

Route requests to different target groups based on URL paths.

🔹 Rule 1 — App1 Routing

Setting	Value
Condition Type	Path
Path Value	/app1*
Action	Forward to TG-App1

Setting	Value
Priority	1

🔹 Rule 2 — App2 Routing

Setting	Value
Condition Type	Path
Path Value	/app2*
Action	Forward to TG-App2
Priority	2

📌 Task 9 — Test ALB Path-Based Routing

Objective

Verify that ALB routes traffic correctly.

Test URLs

```
http://ALB-DNS/app1
```

Expected Result

```
Green APP1 page 🟢
```

```
http://ALB-DNS/app2
```

Expected Result

```
Blue APP2 page 🔵
```

📌 Task 10 — Create Target Group for NLB

Objective

Create target group for Network Load Balancer.

Configuration

Setting	Value
Name	TG-NLB
Protocol	TCP
Port	80
Target Type	Instances
Health Check	TCP

Registered Targets

```
EC2-NLB-1
EC2-NLB-2
```

📌 Task 11 — Create Network Load Balancer (NLB)

Objective

Create an Internet-Facing NLB.

Configuration

Setting	Value
Name	My-NLB
Type	Network Load Balancer
Scheme	Internet-Facing
Protocol	TCP
Port	80
Subnets	Public-Subnet-1 & Public-Subnet-2

Default Action

```
Forward traffic to TG-NLB
```

📌 Task 12 — Test NLB

Objective

Verify NLB distributes traffic between servers.

Test URL

```
http://NLB-DNS/
```

Expected Output

Sometimes:

```
NLB Server 1 🔴
```

Sometimes:

```
NLB Server 2 🟣
```

📌 Task 13 — Configure NLB in Front of ALB

🎯 Objective

Configure a **Network Load Balancer (NLB)** to forward traffic to an **Application Load Balancer (ALB)** using the special:

```
Application Load Balancer Target Type
```

This combines:

- NLB → Static IP + High Performance
- ALB → Smart Layer 7 Routing

📌 What is "Application Load Balancer" Target Type?

Definition

The **Application Load Balancer Target Type** is a special target type available in NLB Target Groups where the NLB can directly forward traffic to an ALB.

AWS automatically manages the ALB IP addresses internally.

🔄 Old Method vs New Method

❌ Old Method (Complex)

```
NLB
  |
  ▼
IP Target Group
  |
  ▼
Manually add ALB IP addresses
```

Problems

- ALB IPs change frequently
- Manual updates required
- Needed Lambda automation
- Complex architecture

✅ New Method (Easy)

```
NLB
  |
  ▼
ALB Target Group
  |
  ▼
Application Load Balancer
```

Benefits

- Direct ALB reference
- AWS manages IP changes
- Simple setup
- Production-ready architecture

🛠️ Step 1 — Create Target Group with ALB Type

Navigation

```
EC2 → Target Groups → Create Target Group
```

Configuration

Setting	Value
Target Type	Application Load Balancer
Name	TG-NLB-to-ALB
Protocol	TCP
Port	80
VPC	LB-Practice-VPC

🔹 Health Check Configuration

Setting	Value
Protocol	HTTP
Path	/app1/index.html

🔹 Register Target

Select:

```
My-ALB
```

Port:

```
80
```

Click:

```
Create Target Group
```

📌 Task 14 — Update NLB to Forward Traffic to ALB

🎯 Objective

Modify the existing NLB listener so traffic flows:

```
NLB → ALB → Target Groups → EC2 Instances
```

🛠️ Step 2 — Update Existing NLB

Navigation

```
EC2 → Load Balancers → My-NLB
```

Steps

1. Go to:

   ```
   Listeners Tab
   ```

2. Select:

   ```
   TCP : 80 Listener
   ```

3. Click:

   ```
   Actions → Edit Listener
   ```

4. Change Default Action:

```
    Forward to: TG-NLB-to-ALB
```

5. Save Changes ✅

🧪 Step 3 — Test the Architecture

Test URL

```
http://NLB-DNS/app1
```

Expected Output

```
GREEN APP1 Page 🟢
```

Test URL

```
http://NLB-DNS/app2
```

Expected Output

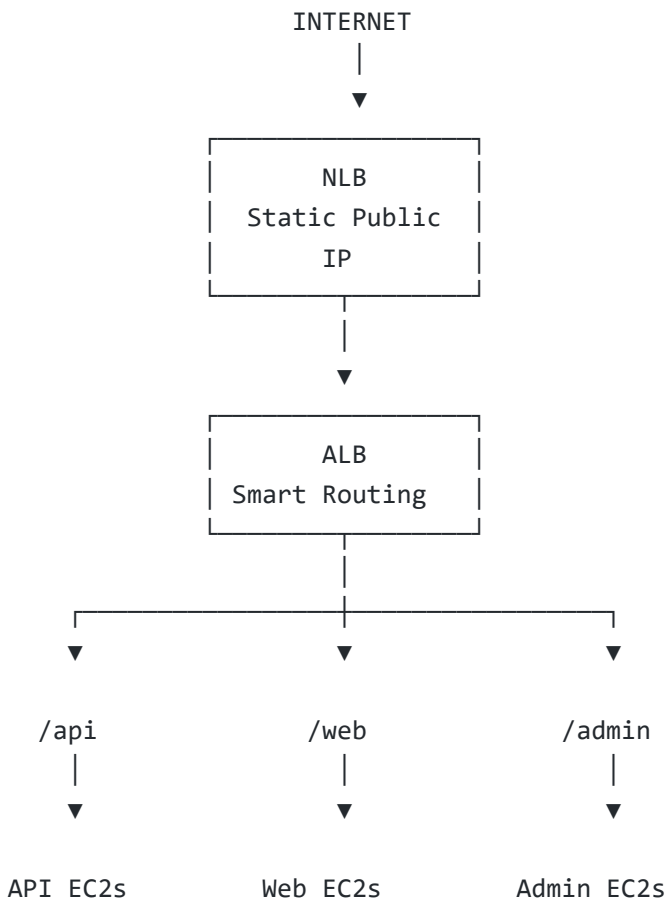
```
BLUE APP2 Page 🔵
```

🔄 Complete Request Flow

```
Step 1:
User sends request to NLB DNS
            |
            ▼

Step 2:
NLB receives TCP traffic
            |
            ▼
```

```
Step 3:
NLB forwards traffic to ALB
(AWS internally manages ALB IPs)
            |
            ▼

Step 4:
ALB reads HTTP request content
            |
            ▼

Step 5:
ALB applies path-based routing
            |
            ├── /app1 → TG-App1 → EC2-App1 🟢
            |
            └── /app2 → TG-App2 → EC2-App2 🔵
```

🌍 Real World Architecture



🔥 Why This Architecture is Powerful

NLB Feature	Benefit
Static IP	Easy firewall whitelisting
High Performance	Handles millions of requests
Ultra Low Latency	Fast traffic forwarding

ALB Feature	Benefit
Path-Based Routing	Smart traffic distribution
Host-Based Routing	Domain-based routing
HTTP Awareness	Understands web requests

🏢 Real World Use Cases

This architecture is commonly used in:

- Banking Applications
- Government Systems
- Enterprise SaaS Platforms
- Healthcare Applications
- B2B APIs
- Security-Sensitive Systems

📌 Why Companies Use NLB in Front of ALB

Because NLB provides:

✅ Static IP
✅ Elastic IP support
✅ IP Whitelisting

✅ High throughput
✅ Low latency

And ALB provides:

✅ Smart Layer 7 routing
✅ Path-based routing
✅ Host-based routing
✅ Header-based routing
✅ WebSocket/gRPC support

✅ Successfully implemented:

```
NLB → ALB → EC2 Architecture
```

with:

- Static Public Entry Point
- Advanced Routing
- High Availability
- Scalable Infrastructure
- Production-Level AWS Design
- Application Load Balancer (ALB)
- Path-Based Routing
- Network Load Balancer (NLB)
- Multi-Server Traffic Distribution
- Apache Web Hosting using User Data

AWS ALB Routing Methods Practice Tasks

📌 Task 1 — Launch EC2-App1 (Host-Based Routing)

Objective

Server for:

```
app1.wellnest-project.online
```

Configuration

Setting	Value
Name	EC2-App1
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#4CAF50; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
🟢 APP 1 Server
</h1>

<h2 style='color:white;'>
```

```
Host: app1.wellnest-project.online
</h2>

</body>
</html>
EOF
```

📌 Task 2 — Launch EC2-App2 (Host-Based Routing)

Objective

Server for:

```
app2.wellnest-project.online
```

Configuration

Setting	Value
Name	EC2-App2
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2
```

```
cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#2196F3; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
🔵 APP 2 Server
</h1>

<h2 style='color:white;'>
Host: app2.wellnest-project.online
</h2>

</body>
</html>
EOF
```

📌 Task 3 — Launch EC2-API (Path-Based Routing)

Objective

Server for:

```
/api
```

Configuration

Setting	Value
Name	EC2-API
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/api

cat <<EOF > /var/www/html/api/index.html
<html>
<body style='background-color:#FF9800; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
🟠 API Server
</h1>

<h2 style='color:white;'>
Path: /api
</h2>

</body>
</html>
EOF
```

📌 Task 4 — Launch EC2-Web (Path-Based Routing)

Objective

Server for:

```
/web
```

Configuration

Setting	Value
Name	EC2-Web
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

mkdir -p /var/www/html/web

cat <<EOF > /var/www/html/web/index.html
<html>
<body style='background-color:#9C27B0; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
🟣 Web Server
</h1>

<h2 style='color:white;'>
Path: /web
</h2>

</body>
</html>
EOF
```

📌 Task 5 — Launch EC2-Mobile (Query String Routing)

Objective

Server for:

```
?platform=mobile
```

Configuration

Setting	Value
Name	EC2-Mobile
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#E91E63; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
📱 Mobile Server
</h1>

<h2 style='color:white;'>
You are on Mobile Platform!
</h2>

<h3 style='color:white;'>
Routed via Query String
```

```
</h3>

</body>
</html>
EOF
```

📌 Task 6 — Launch EC2-Desktop (Query String Routing)

Objective

Server for:

```
?platform=desktop
```

Configuration

Setting	Value
Name	EC2-Desktop
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2
```

```
cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#607D8B; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
🖥️ Desktop Server
</h1>

<h2 style='color:white;'>
You are on Desktop Platform!
</h2>

<h3 style='color:white;'>
Routed via Query String
</h3>

</body>
</html>
EOF
```

📌 Task 7 — Create Target Groups

Target Groups Created

Target Group	Purpose
TG-App1	Host-Based Routing
TG-App2	Host-Based Routing
TG-API	Path-Based Routing
TG-Web	Path-Based Routing
TG-Mobile	Query String Routing
TG-Desktop	Query String Routing

📌 Task 8 — Create Application Load Balancer

Configuration

Setting	Value
Name	My-ALB
Type	Internet-Facing
Protocol	HTTP
Port	80
Security Group	LB-SG

Default Action

```
Forward to TG-Desktop
```

📌 Task 9 — Configure Route 53 DNS Records

DNS Records Created

Record Name	Points To
wellnest-project.online	My-ALB
app1.wellnest-project.online	My-ALB
app2.wellnest-project.online	My-ALB

📌 Task 10 — Configure Host-Based Routing Rules

🔹 Rule 1 — App1

Setting	Value
Host Header	app1.wellnest-project.online

Setting	Value
Action	Forward to TG-App1
Priority	1

🔹 Rule 2 — App2

Setting	Value
Host Header	app2.wellnest-project.online
Action	Forward to TG-App2
Priority	2

📌 Task 11 — Configure Path-Based Routing Rules

🔹 API Rule

Setting	Value
Path	/api*
Action	Forward to TG-API
Priority	3

🔹 Web Rule

Setting	Value
Path	/web*
Action	Forward to TG-Web
Priority	4

📌 Task 12 — Configure Query String Routing Rules

🔹 Mobile Query Rule

Setting	Value
Query Key	platform
Query Value	mobile
Action	Forward to TG-Mobile
Priority	7

🔹 Desktop Query Rule

Setting	Value
Query Key	platform
Query Value	desktop
Action	Forward to TG-Desktop
Priority	8

📌 Task 13 — Launch EC2-GET (HTTP Method Routing)

Objective

Handle GET requests.

Configuration

Setting	Value
Name	EC2-GET
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-1
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#00BCD4; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
📖 GET Server - Read Operations
</h1>

<h2 style='color:white;'>
You sent a GET Request!
</h2>

<h3 style='color:white;'>
This server handles READ operations only
</h3>

</body>
</html>
EOF
```

📌 Task 14 — Launch EC2-POST (HTTP Method Routing)

Objective

Handle POST requests.

Configuration

Setting	Value
Name	EC2-POST
AMI	Ubuntu
Instance Type	t2.micro
Subnet	Public-Subnet-2
Security Group	EC2-SG

User Data

```
#!/bin/bash

apt update -y
apt install apache2 -y

systemctl start apache2
systemctl enable apache2

cat <<EOF > /var/www/html/index.html
<html>
<body style='background-color:#F44336; text-align:center;'>

<h1 style='color:white; margin-top:200px;'>
🖍️ POST Server - Write Operations
</h1>

<h2 style='color:white;'>
You sent a POST Request!
</h2>

<h3 style='color:white;'>
This server handles WRITE operations only
</h3>

</body>
```

```
</html>
EOF
```

📌 Task 15 — Create Target Groups for HTTP Method Routing

Target Group	Purpose
TG-GET	GET Requests
TG-POST	POST Requests

📌 Task 16 — Configure HTTP Method Routing Rules

🔹 GET Rule

Setting	Value
HTTP Method	GET
Action	Forward to TG-GET
Priority	9

🔹 POST Rule

Setting	Value
HTTP Method	POST
Action	Forward to TG-POST
Priority	10

📌 Task 17 — Test Host-Based Routing

Test URLs

```
http://app1.wellnest-project.online
```

Expected:

```
GREEN APP1 Page 🟢
```

```
http://app2.wellnest-project.online
```

Expected:

```
BLUE APP2 Page 🔵
```

📌 Task 18 — Test Path-Based Routing

Test URLs

```
http://wellnest-project.online/api
```

Expected:

```
ORANGE API Page 🟠
```

```
http://wellnest-project.online/web
```

Expected:

```
PURPLE WEB Page 🟣
```

📌 Task 19 — Test Query String Routing

Test URLs

```
http://wellnest-project.online?platform=mobile
```

Expected:

```
PINK Mobile Page 📱
```

```
http://wellnest-project.online?platform=desktop
```

Expected:

```
GREY Desktop Page 🖥️
```

📌 Task 20 — Test HTTP Method Routing

Test GET Request

```
curl -X GET http://wellnest-project.online
```

Expected:

```
GET Server Page 📖
```

Test POST Request

```
curl -X POST http://wellnest-project.online
```

Expected:

```
POST Server Page 🖍️
```

✅ Routing Methods Practiced

Routing Type	Status
Host-Based Routing	✅ Completed
Path-Based Routing	✅ Completed
Query String Routing	✅ Completed
HTTP Method Routing	✅ Completed

Successfully implemented:

- ✅ Host-Based Routing
- ✅ Path-Based Routing
- ✅ Query String Routing
- ✅ HTTP Method Routing
- ✅ Route 53 + ALB Integration
- ✅ Multi-Target Group Architecture